Report 2 Quantification of the Aligments

Ismael Maximiliano De Los Santos Huesca

2026-04-16

Quantification of the Aligments

Transciptomics

De Los Santos Huesca Ismael Maximiliano

Dr. David Valle

Fecha: 16 April 2026

Table of Contents

1 Pre-proccesing previous alignments **2**

2 PseudoAligners **2**

2.1 Salmon . . . 2

2.2 Kallisto . . . 2

2.3 Run . . . 5

2.3.1 Parse results . . . 5

2.4 Analysis . . . 6

3 Big-wig files **7**

3.1 Results . . . 7

4 Quantification **8**

4.0.1 Count Matrices . . . 10

4.0.2 Cleaning the featureCounts Matrices . . . 10

4.0.3 Hisat . . . 11

4.0.4 star . . . 11

4.0.5 salmon . . . 11

4.0.6 kallisto . . . 12

5 Discussion **13**

6 Code **13**

1 Pre-proccesing previous alignments

Once we have donde the aligments we can made the filtering of those with samtools especially with the `samtools view` command, this command allows us to filter the alignments based on different criteria like the mapping quality, the flag of the alignment, etc.

```
# iterate over all th e SRR ids to run de corresponding alingments
for srr in SRR_ids:
    sp.run(
        ["qsub", "src/jdl/clean_alignments.jdl", "-s", srr, "-p", "results/alignments"]
    )
```

```
There are 8 hisat clean unpaired alignments 8 hisat clean paired alignments
There are 8 star clean unpaired alignments 8 star clean paired alignments
There are 8 hisat clean unpaired indexed alignments 8 hisat clean paired indexed alignments
There are 8 star clean unpaired indexed alignments 8 star clean paired indexed alignments
```

2 PseudoAligners

Once the alignment the next step is the quantification of the reads this can be assed by tools like `featureCounts` from Subread this generates a table file with the counts for each gene in function of a reference genome annotation (GFF or GTF file) and the `bam` files generated for a common aligner, this generates a table of counts that is the milestone for the diferential expression analysis, but there is another approach in where the bam files are not generated and even the alignment euristic does not is follow in a traditional way.

In the **pseudoAligmnets** the alogorithms, on big terms, use a de brujin graph to represent the reference genome as transcripts and the reads try to follow the paths on the possible transcripts based in a probability model, in wich the transcipt with the best probability is the one that is selected for the quantification of the reads, this has a upgrade in the time and resources used for the quantification and also reported a correlation near to 1 with the aligment approach, also it does not has to procces the bam files for quntification using `featureCounts` or similar tools, so it is a more direct approach for the quantification of the reads, they use tools like tximport for merging all the counts made by tools like kallisto or salmon in a single table for the following analysis.

2.1 Salmon

Salmon is a pseudoaligner that could make both a `mapping-based` quantification and a `aligment-based` quantification, in this analysis we are going to use the `mapping-based` approach that is the one that use the **quasi-mapping** algorithm and is diferent to the traditional time consuming aligment approach.

> This indexes were provided so the salmon index procedure were not neccesary

2.2 Kallisto

In the case of Kallisto pseudo aligner we have to generate an index because it have not been provided by the class so we have to generate it using a multifasta file with al the transcriptome of the genome version we are working with, in this case the m36 version of the mouse genome, we can obtain this multifasta file from genecode and then generate the index with the `kallisto index` command.

```
# | eval: false

#first we obtain the multifasta file with the transcriptome of the m36 version of the mouse genome
wget https://ftp.ebi.ac.uk/pub/databases/\
gencode/Gencode_mouse/release_M36/gencode.vM36.transcripts.fa.gz \
-O data/indexes/mouse_m36_transcriptome.fa.gz
#now we can download the gtf file with the annotation
wget https://ftp.ebi.ac.uk/pub/databases/\
gencode/Gencode_mouse/release_M36/gencode.vM36.chr_patch_hapl_scaff.annotation.gtf.gz \
-O data/indexes/mouse_m36_annotation.gtf.gz
#finally the comopresive genome fasta file
wget https://ftp.ebi.ac.uk/pub/databases/\
gencode/Gencode_mouse/release_M36/GRCm39.genome.fa.gz \
-O data/indexes/mouse_m36_genome.fa.gz
#generate the directory for kallisto indexes
mkdir -p data/indexes/kallisto/
#generate the directory for kb indexes
mkdir -p data/indexes/kb/

#now we generate the index with the kallisto index command
conda run -n kallisto kallisto index \
-i data/indexes/kallisto/mouse_m36_transcriptome.idx data/indexes/mouse_m36_transcriptome.fa.gz \
-t 4

#now we generate the index with the kallisto index command
conda run -n bio_informatics kb ref -i data/indexes/kb/mouse_m36_transcriptome.idx \
-g data/indexes/kb/mouse_m36_t2g.txt -t 8 \
-f1 data/indexes/mouse_m36_transcriptome.fa.gz \
data/indexes/mouse_m36_genome.fa.gz data/indexes/mouse_m36_annotation.gtf.gz
```

The next step is to build the sample table for the kallisto quantification, this is a table with the sample names and the paths to the fastq files for each sample, this is necessary for the `kallisto quant` command that is the one that make the quantification of the reads, then we can run the `kallisto quant` program for all the samples.

	sample	fastq_1	fastq_2
1	Bone_1	data/SRR9127010/SRR9127010_1_clean.fastq	data/SRR9127010/SRR9127010_2_clean.fastq
2	Bone_1	data/SRR9127296/SRR9127296_1_clean.fastq	data/SRR9127296/SRR9127296_2_clean.fastq
4	Bone_1	data/SRR9127157/SRR9127157_1_clean.fastq	data/SRR9127157/SRR9127157_2_clean.fastq
5	Bone_1	data/SRR9127369/SRR9127369_1_clean.fastq	data/SRR9127369/SRR9127369_2_clean.fastq
0	Bone_24	data/SRR9126934/SRR9126934_1_clean.fastq	data/SRR9126934/SRR9126934_2_clean.fastq
3	Bone_24	data/SRR9126859/SRR9126859_1_clean.fastq	data/SRR9126859/SRR9126859_2_clean.fastq
6	Bone_24	data/SRR9127396/SRR9127396_1_clean.fastq	data/SRR9127396/SRR9127396_2_clean.fastq
7	Bone_24	data/SRR9127293/SRR9127293_1_clean.fastq	data/SRR9127293/SRR9127293_2_clean.fastq

	sample	fastq_1
1	Bone_1	data/SRR9127010/SRR9127010_1_clean.fastq
2	Bone_1	data/SRR9127296/SRR9127296_1_clean.fastq
4	Bone_1	data/SRR9127157/SRR9127157_1_clean.fastq
5	Bone_1	data/SRR9127369/SRR9127369_1_clean.fastq
0	Bone_24	data/SRR9126934/SRR9126934_1_clean.fastq
3	Bone_24	data/SRR9126859/SRR9126859_1_clean.fastq
6	Bone_24	data/SRR9127396/SRR9127396_1_clean.fastq
7	Bone_24	data/SRR9127293/SRR9127293_1_clean.fastq

For using the tables with the metadata we ares using the kb prgrams from the suite kallisto-bustools, this suite have a program called `kb ref` that allows us to generate the index for the quantification and also the table with the annotation of the genes for the following analysis, this is something that we have to do before running the quantification with kallisto.

```
#| eval: false

#output directory for the kb indexes
output_dir="results/pseudo_alignments/kb/paired"
mkdir -p $output_dir

#run kallisto for paired data
echo "conda run -n bio_informatics kb count \
-i data/indexes/kb/mouse_m36_transcriptome.idx \
-g data/indexes/kb/mouse_m36_t2g.txt \
-x BULK --parity paired -o $output_dir --bootstraps 100 \
--tmp kb_paired -t 8 -m 12G \
data/indexes/kallisto/paired_kallisto_samples.csv"|\
qsub \
-N "kb_PR" \
-cwd \
-o "${output_dir}_kbPR.out" \
-e "${output_dir}_kbPR.err" \
-q default \
-pe smp 8 \
-l h_rt=04:00:00 \
-l h_vmem=16G

#=====================================================================
#unpaired
#=====================================================================

output_dir="results/pseudo_alignments/kb/unpaired"
mkdir -p $output_dir

#run kallisto for unpaired data
echo "conda run -n bio_informatics kb count \
-i data/indexes/kb/mouse_m36_transcriptome.idx \
-g data/indexes/kb/mouse_m36_t2g.txt \
-x BULK --parity single -o $output_dir --bootstraps 100 --tmp kb_unpaired \
-t 8 -m 12G \
data/indexes/kallisto/unpaired_kallisto_samples.csv"|\
qsub \
-N "kb_UN" \
-cwd \
-o "${output_dir}_kbUN.out" \
-e "${output_dir}_kbUN.err" \
-q default \
-pe smp 8 \
-l h_rt=04:00:00 \
-l h_vmem=16G
```

> The kb pipeline is better for single cell data so for comparison we are using the results from the old kallisto program

2.3 Run

With the sample tables made for the kallisto quantification we can run the `kallisto quant` and `salmon quant` for all the samples, this will generate a table with the counts for each gene in function of the reference genome annotation and the bam files generated for a common aligner, this generates a table of counts that is the milestone for the diferential expression analysis.

```
# first we generate the temporary scripts for the salmon quantification
sp.run(["bash", "src/Generate_tempsPseudo.sh"])

# now we can make the aligment for each sample
for srr in SRR_ids:
    sp.run(["qsub", "src/jdl/pseudo_align.jdl", "-S", srr])
```

> Note here both pseudo aligners were run with 4 threads and 12 gigabytes of memory, also because both pseudo-aligners support boopstrapping for abundance estimation the number of boostrappings were set in 100, it is also important to note that in salmon the options –gcBias, –seqBias, –posBias, –validateMappings, –rangeFactorizationBins(in 4), were set.

2.3.1 Parse results

With all the pseudo aligments made now we can asses the clean a parsed of both results and log files, this, essentially, is for deleting all the intermate bootstrapping files that kallist make and for parsing the stadistics from the pseudoaligment, those like oveall alignment, time and what pseudo-aligner done it.

```
#| eval: false

#Run the script for pasing the results
bash src/parsed_pseudo.sh -p results/pseudo_alignments
```

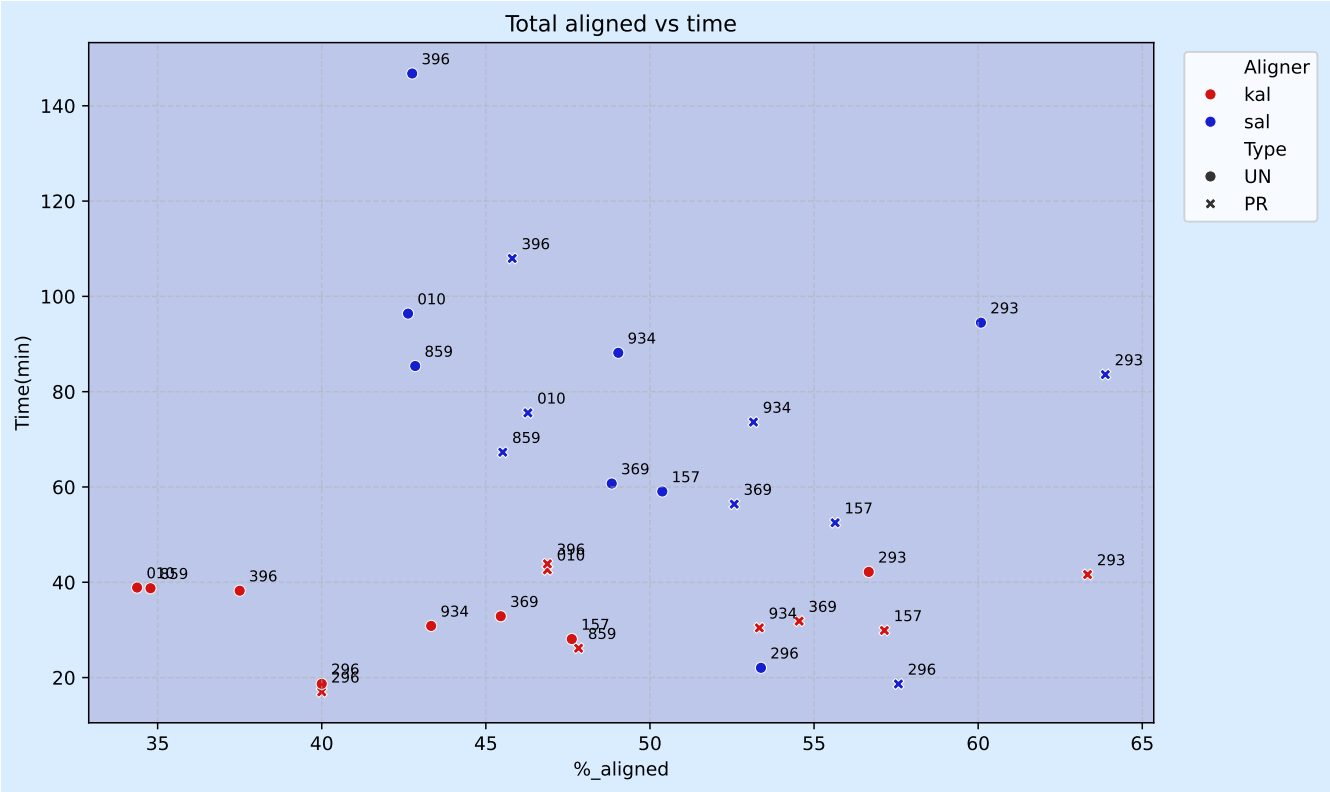
Table 3

SRR_id	Aligner	Type	%_aligned	Time(min)
SRR9126859	kal	UN	34.783	38.746
SRR9126859	sal	PR	45.516	67.297
SRR9126859	sal	UN	42.847	85.386
SRR9126859	kal	PR	47.826	26.156
SRR9126934	kal	PR	53.333	30.432
SRR9126934	kal	UN	43.333	30.844
SRR9126934	sal	PR	53.154	73.621
SRR9126934	sal	UN	49.035	88.155
SRR9127010	kal	PR	46.875	42.615
SRR9127010	kal	UN	34.375	38.893
SRR9127010	sal	PR	46.281	75.548
SRR9127010	sal	UN	42.630	96.385
SRR9127157	kal	PR	57.143	29.893
SRR9127157	kal	UN	47.619	28.086
SRR9127157	sal	PR	55.642	52.493
SRR9127157	sal	UN	50.377	59.055
SRR9127293	kal	PR	63.333	41.622
SRR9127293	kal	UN	56.667	42.176
SRR9127293	sal	PR	63.874	83.593
SRR9127293	sal	UN	60.084	94.484
SRR9127296	kal	PR	40.000	17.001
SRR9127296	kal	UN	40.000	18.686
SRR9127296	sal	PR	57.569	18.654
SRR9127296	sal	UN	53.382	22.047
SRR9127369	kal	PR	54.545	31.843

SRR_id	Aligner	Type	%_aligned	Time(min)
SRR9127369	kal	UN	45.455	32.870
SRR9127369	sal	PR	52.568	56.402
SRR9127369	sal	UN	48.835	60.725
SRR9127396	kal	PR	46.875	43.866
SRR9127396	kal	UN	37.500	38.232
SRR9127396	sal	PR	45.804	107.960
SRR9127396	sal	UN	42.756	146.770

2.4 Analysis

With the results that we have parsed we can make comparisons between both aligners and their times of run.



We can see in this plot that the general tendency of the results is that, as with the traditional aligners, the time used to align, or in this case "pseudo-align", paired data is greater than that used for single-end data. This difference is larger for `salmon`.

Another tendency shared with traditional aligners is that the alignment rate is higher in the single-end data.

It is important to note that `salmon` has a more time-consuming performance. This is due to the time required for correcting biases (`--gcBias`, `--seqBias`, `--posBias`) and for improving abundance estimates (`--validateMappings`, `--rangeFactorizationBins`). We can see that this is worthwhile for improving the alignment rate in the case of the options that correct biases in the reads (further analysis would be needed to evaluate the impact of the abundance estimation options).

3 Big-wig files

Another feature than can be analiced by the results of the aligments are the bigwig files, these can be generated by the `bamCoverage` program from the deeptools suite, this program allows us to generate bigwig files from the bam files generated by the aligners, this bigwig files can be visualized in genome browsers like IGV, UCSC Genome Browser or IGB and can give us a visual representation of the coverage of the reads in the genome, this can be useful for seeing the quality of the alignment and also for seeing if there are any regions of the genome that are not being covered by the reads.

This can be useful for seeing the overall diference between samples and between the coditions that we are trying to elucidate the expression diferences for that we are going to use only the paired end data that we have generated for both traditional aligners (`star` and `hisat`) this because are those that generate bam files.

```
# run the bigwig processing for every SRR id
for srr in SRR_ids:
    sp.run(["qsub", "src/jdl/run_bamCov.jdl", "-p", "results/alignments/clean/", "-s", srr])
```

3.1 Results

For this brief analysis with the `IGB` genome browser, we have arbitrarily selected two samples from both conditions tested in our tissue (1 and 24 months). These are `SRR9126934` for 24 months and `SRR9127010` for 1 month. With these samples, we searched in the genome browser for the positions related to the gene **Ccl8**, referring to the cytokine that is associated with inflammation, which the authors in the original paper relate to the progression of life.

	gene_id	trans_id	gene_name	isoform	chromosome	start	end
172921	ENSMUST00000009329	ENSMUSG00000009185	Ccl8	Ccl8-201	chr11	82006011	82007625

Figure 1: Ccl8_genomeBW_hi_sr

We can see that the expression of the gene Ccl8 is consistent with the data reported by the authors, it is important to note that thi is not a concluding result because here we are only viewing that the normalization a semi-quantification that bamCovarage do capture the difference acrros this 2 samples, and that diference is consitent in the data proccesed with `hisat2` and `star`, so this is a good insight for the next steps.

4 Quantification

Once we have all the aligments and pseudoaligments done we can start assesing to construct the famous **count matrix** in wich the rows are the genes reported in our reference genome annotation (normally a `GTF` file) and the columns are the samples that we have in our experiment, this count matrix is totally made by `kb` (kallisto-bustools), for salmon is partially made because it uses uniques fastq files (paired or unpaired) but because it *maps* the reads to the transcriptome, those genes are already quantified so we only have to merge the tables generated for each sample in a single table with all the samples, this is something that is done by the `tximport` package in R.

But for the case of the traditional aligners we have to use the `featureCounts` program from the Subread suite, this program allows us to generate a count matrix from the bam files generated by the aligners and the reference genome annotation (GTF file) and the bam files generated by the alignment of each sample to the reference genome, in the case of our analysis we have made 2 diferent aligments types (**paired** and **unpaired**) for each aligner (`hisat2` and `star`) having 4 diferent count matrixes to generate with featureCounts, this plus the ones that were made by by pseudoaligners.

Aligner	type	number_samples	results_path
hisat	paired	8	results/alignments/clean/*HIpr_clean.bam
hisat	unpaired	8	results/alignments/clean/*HIun_clean.bam
star	paired	8	results/alignments/clean/*SRpr_clean.bam
star	unpaired	8	results/alignments/clean/*SRun_clean.bam
salmon	paired	8	results/pseudo_alignments/salmon/*salPR
salmon	unpaired	8	results/pseudo_alignments/salmon/*salUN
kallisto	paired	8	results/pseudo_alignments/kb/paired
kallisto	unpaired	8	results/pseudo_alignments/kb/unpaired

4.0.0.1 FeatureCounts

```
#| eval: false

# Run the feature counts for hisat2 and star for paired and unpaired data
bash src/run_featureC.sh -p results/alignments/clean/ -g data/indexes/mouse_m36_annotation.gtf.gz
```

4.0.0.2 Tximport

```
# | eval: false

#import the libraries
library(tximport)
library(readr)

#load the required tables for the tximport
t2g_df <- read.table("data/indexes/kb/mouse_m36_t2g.txt", sep = "\t")
#keep only the columns t2g that matters for tximport (transcript id and gene id)
trans_2_gen_tx <- t2g_df[1:2]
#clean those columns
trans_2_gen_tx$V1 <- gsub("\\..*", "", trans_2_gen_tx$V1)
trans_2_gen_tx$V2 <- gsub("\\..*", "", trans_2_gen_tx$V2)

metadata_df <- read.table(
  "data/Meatadata_male_bone_1_24M.tsv",
  header = TRUE,
  sep = "\t"
)


for (aligner in c("salmon", "kallisto")) {
  #asign the file with the counts for each aligner
  if (aligner == "salmon") {
    count_file <- "quant.sf"
  } else {
    count_file <- "bs_abundance_99_clean.tsv"
  }
  #extract aligner abreviation
  align_sub <- substr(aligner, 1, 3)
  #create the vector with the paths for the results
  paired_files <- file.path(
    "results/pseudo_alignments",
    aligner,
    paste0(metadata_df$srr_id, paste0("_", align_sub, "PR")),
    count_file
  )
  single_files <- file.path(
    "results/pseudo_alignments",
    aligner,
    paste0(metadata_df$srr_id, paste0("_", align_sub, "UN")),
    count_file
  )
  #out the respectuve names to the vector
  sample_names <- paste0(metadata_df$name, "_", metadata_df$months)
  names(paired_files) <- sample_names
  names(single_files) <- sample_names
  #merge the salmon quantification results
  txi_paired <- tximport(paired_files, type = aligner, tx2gene = trans_2_gen_tx)
  txi_unpaired <- tximport(
    single_files,
    type = aligner,
    tx2gene = trans_2_gen_tx
  )
  #save the counts matrix
```

```
  write.csv(
    txi_paired,
    file.path(
      "results/count_mtr",
      paste0(align_sub, "_", "pr", "countMtr.txt")
    ),
    sep = "\t"
  )
  write.csv(
    txi_unpaired,
    file.path(
      "results/count_mtr",
      paste0(align_sub, "_", "un", "countMtr.txt")
    ),
    sep = "\t"
  )
}
```

4.0.1 Count Matrices

With all the previous steps, we generated the count matrices. These are 8 in total, 2 for each aligner or pseudo-aligner (`hisat2`, `star`, `salmon`, `kallisto`), which are visualized below:

4.0.2 Cleaning the featureCounts Matrices

The tables generated with `featureCounts` have a comment line, and the columns that contain the counts are labeled with the file path they come from. Therefore, we are going to clean them.

```
#| eval: false

#clean for every SR or HI mtrx
for mtrx in results/count_mtr/[SRHI]*_*countMtr.txt; do
    #make the output name
    out_file=${mtrx/.*/"_clean.txt"}
    #clean the matrix
    awk 'BEGIN{FS=OFS="\t"};
        NR == 1{next}
        NR==2 {printf "%s%s", $1,"\t";
            for(i=7;i<=14;i++){
                sub(/.*\//, "", $i)
                sub(/_.*/,"",$i)
                printf "%s%s", $i,(i<14 ? "\t" : "\n")
            }
        } NR != 2 {print($1,$7,$8,$9,$10,$11,$12,$13,$14)}' "$mtrx" > $out_file
done
```

4.0.3 Hisat

Table 6: Count matrix generated by hisat2 with paired-end data

	SRR9126859	SRR9126934	SRR9127010	SRR9127157	SRR9127293	SRR9127296	SRR91
ENSMUSG00000102693.2	31	20	55	9	10	4	0
ENSMUSG00000064842.3	1	1	0	0	3	1	0
ENSMUSG00000051951.6	79	100	121	59	21	8	53
ENSMUSG00000102851.2	4	13	20	2	2	1	8
ENSMUSG00000103377.2	32	21	44	0	13	5	9

Table 7: Count matrix generated by hisat2 with single-end data

	SRR9126859	SRR9126934	SRR9127010	SRR9127157	SRR9127293	SRR9127296	SRR91
ENSMUSG00000102693.2	11	10	29	6	5	2	0
ENSMUSG00000064842.3	0	0	0	0	2	0	0
ENSMUSG00000051951.6	41	46	63	32	12	3	28
ENSMUSG00000102851.2	3	1	10	1	1	1	4
ENSMUSG00000103377.2	15	10	24	0	5	3	5

4.0.4 star

Table 8: Count matrix generated by star with paired-end data

	SRR9126859	SRR9126934	SRR9127010	SRR9127157	SRR9127293	SRR9127296	SRR91
ENSMUSG00000102693.2	31	20	55	9	10	4	0
ENSMUSG00000064842.3	1	1	0	0	3	1	0
ENSMUSG00000051951.6	79	100	121	59	21	8	53
ENSMUSG00000102851.2	4	13	20	2	2	1	8
ENSMUSG00000103377.2	32	21	44	0	13	5	9

Table 9: Count matrix generated by star with single-end data

	SRR9126859	SRR9126934	SRR9127010	SRR9127157	SRR9127293	SRR9127296	SRR91
ENSMUSG00000102693.2	11	10	29	6	5	2	0
ENSMUSG00000064842.3	0	0	0	0	2	0	0
ENSMUSG00000051951.6	41	46	63	32	12	3	28
ENSMUSG00000102851.2	3	1	10	1	1	1	4
ENSMUSG00000103377.2	15	10	24	0	5	3	5

4.0.5 salmon

Table 10: Count matrix generated by salmon with paired-end data

	Bone_8_24	Bone_16_1	Bone_12_1	Bone_21_24	Bone_46_1	Bone_32_1	Bone_11_2
ENSMUSG00000000001	1193.9	811.9	227.0	647.0	685.0	845.9	1000.8
ENSMUSG00000000003	0.0	6.0	0.0	1.0	3.0	1.0	5.0
ENSMUSG00000000028	4.0	9.0	0.0	6.0	0.0	4.0	20.3
ENSMUSG00000000031	77.0	101.0	6.0	15.0	112.0	144.7	13.8
ENSMUSG00000000037	14.0	50.6	3.0	13.3	2.3	19.0	34.3

Table 11: Count matrix generated by salmon with single-end data

	Bone_8_24	Bone_16_1	Bone_12_1	Bone_21_24	Bone_46_1	Bone_32_1	Bone_11_2
ENSMUSG00000000001	1181.9	797.8	228.0	639.9	678.6	842.4	978.3
ENSMUSG00000000003	0.0	4.0	0.0	1.0	3.0	0.0	0.0
ENSMUSG00000000028	3.0	4.1	0.0	8.0	0.0	2.3	12.8
ENSMUSG00000000031	72.0	99.0	8.0	15.0	111.0	104.9	15.4
ENSMUSG00000000037	12.0	50.4	0.0	0.0	0.0	8.9	0.0

4.0.6 kallisto

Table 12: Count matrix generated by kallisto with paired-end data

	Bone_8_24	Bone_16_1	Bone_12_1	Bone_21_24	Bone_46_1	Bone_32_1	Bone_11_2
ENSMUSG00000000001	1210.0	809.0	225.0	670.0	682.9	837.9	1039.6
ENSMUSG00000000003	0.0	5.0	0.0	4.0	6.0	3.0	12.0
ENSMUSG00000000028	246.6	198.5	77.0	188.1	177.4	209.2	294.8
ENSMUSG00000000031	79.0	107.0	9.0	17.0	94.0	268.0	40.0
ENSMUSG00000000037	39.0	76.0	3.0	28.0	17.0	25.0	57.0

Table 13: Count matrix generated by kallisto with single-end data

	Bone_8_24	Bone_16_1	Bone_12_1	Bone_21_24	Bone_46_1	Bone_32_1	Bone_11_2
ENSMUSG00000000001	1173.7	743.7	235.0	638.0	644.5	836.6	1010.4
ENSMUSG00000000003	0.0	5.0	0.0	6.0	6.0	0.0	0.0
ENSMUSG00000000028	273.7	198.8	87.0	156.4	193.2	186.3	286.8
ENSMUSG00000000031	63.0	112.0	9.0	15.0	107.0	262.0	20.0
ENSMUSG00000000037	18.0	64.0	2.0	16.0	5.0	21.0	27.0

5 Discussion

Here we made the next step of the RNA-seq data processing. The quantification step uses the results from the previous alignments in order to obtain the expression levels that are relative to each sample. This is crucial because in this step we do not only have to condensate all the information stored in the BAM files, we also have to make sure that all that information is concordant with the genome we have indexed and also with its annotation. Here, the genomic positions that were the result of the alignments start to make sense in that they now have a gene or transcript name as a product of the combination of three essential information sources: the **BAM** files, the **FASTA** with the transcriptome, and the **GTF** file with the annotation of all those transcripts reported in the genome. With the tool `featureCounts` from the `Subread` package, we can combine these to generate the count matrices with the traditional alignment procedure.

Another approximation that we explore in this report is the **pseudo-alignment** approach, in which we use two alignment tools, `salmon` and `kallisto`. These have different optimization procedures but are based on the same de Bruijn graph approximation, in which the search space is reduced dramatically by searching only in the transcriptome. One of the advantages of this procedure is that it does not generate a **BAM** file; instead, because their indexes already contain the annotation, they generate an analogous sample-specific count matrix that can be merged with all the other samples' count matrices using tools like `tximport`. One disadvantage of this approach is that it does not have the possibility to uncover expression of transcripts that are not reported in the reference data. This is supposed to be balanced by the fact that this approach tends to be faster, something that we do not observe in the time consumption of both algorithms. It is important to note that this may be due to the use of bootstrapping and `salmon`-specific options that maximize not only resource consumption but also time, resulting in similar scales compared with the traditional aligners. Additionally, we can note that the alignment rate is lower than that obtained with `hisat2` and `STAR` (traditional aligners), likely due to the reduced search space.

Finally, it is interesting to observe, in a preliminary way, the differences between samples and between experimental conditions using a **genome browser**, obtained from processing the **BAM** files together with the annotation files. This is a good approximation to generate hypotheses about possible real differences between conditions, but it is not, in any way, conclusive evidence of biological differences between them. More sophisticated statistical modeling is needed to assess this, which will be addressed in further differential expression analysis.

6 Code

All the code and extended results are disponible in Tabula_Muris_Transcriptomic_analysis git hub repository